

Formal Methods and Specification (SS 2025/26)

Lecture 5:

Correctness of Programs with Control Structures

Stefan Ratschan

Czech Technical University in Prague



Evropský sociální fond Praha & EU: Investujeme do vaší budoucnosti

Complexity Reduction

software systems

...

control structures

basic paths:

- assume** $x \geq 5$
- $y \leftarrow x$
- $z \leftarrow x + y$
- @ $z \geq 0$

logic, proofs

Control Structures: Example:

if $x > y$ **then**

$r \leftarrow x$

else

$r \leftarrow y$

@ $r = \max\{x, y\}$

Two execution paths:

$r \leftarrow x$

@ $r = \max\{x, y\}$

$r \leftarrow y$

@ $r = \max\{x, y\}$

Executed only if certain conditions hold.

assume $x > y$

$r \leftarrow x$

@ $r = \max\{x, y\}$

assume $\neg x > y$

$r \leftarrow y$

@ $r = \max\{x, y\}$

Control Structures: Example:

if $x > y$ **then**

$r \leftarrow x$

else

$r \leftarrow y$

@ $r = \max\{x, y\}$

Corresponding basic paths:

assume $x > y$

$r \leftarrow x$

@ $r = \max\{x, y\}$

assume $\neg x > y$

$r \leftarrow y$

@ $r = \max\{x, y\}$

Corresponding verification conditions:

$$[x > y \wedge r = x] \Rightarrow r = \max\{x, y\}$$

$$[\neg x > y \wedge r = y] \Rightarrow r = \max\{x, y\}$$

General Idea: Linearization of Control Structures

programs with control structures $\rightarrow$ basic paths

Definition (preliminary version):

A basic path is a *basic path of a program P* iff
the basic path corresponds to
a sequence of program lines of P from one assertions to another one, with
control structures replaced by **assume** statements

(precise version later)

for **if-then-else**, **while**, **for** ...

Linearization of **if-then-else**

if P **then**

...

else

...

Linearization:

- ▶ path following **if**: **assume** P
- ▶ path following **else**: **assume** $\neg P$

Basic Paths of **if-then-else** Cascades

if $x_1 > y_1$ **then**

$r_1 \leftarrow x_1$

else

$r_1 \leftarrow y_1$

if $x_2 > y_2$ **then**

$r_2 \leftarrow x_2$

else

$r_2 \leftarrow y_2$

if $r_1 > r_2$ **then**

$r \leftarrow r_1$

else

$r \leftarrow r_2$

@ $r = \max\{x_1, x_2, y_1, y_2\}$

Example of basic path:

assume $x_1 > y_1$

$r_1 = x_1$

assume $x_2 > y_2$

$r_2 = x_2$

assume $r_1 > r_2$

$r = r_1$

@ $r = \max\{x_1, x_2, y_1, y_2\}$

$2^3 = 8$ basic paths

Basic Paths of **if-then-else** Cascades

if $x_1 > y_1$ **then**

$r_1 \leftarrow x_1$

else

$r_1 \leftarrow y_1$

@ $r_1 = \max\{x_1, y_1\}$

if $x_2 > y_2$ **then**

$r_2 \leftarrow x_2$

else

$r_2 \leftarrow y_2$

@ $r_1 = \max\{x_1, y_1\}, r_2 = \max\{x_2, y_2\}$

if $r_1 > r_2$ **then**

$r \leftarrow r_1$

else

$r \leftarrow r_2$

@ $r = \max\{x_1, x_2, y_1, y_2\}$

assume $x_1 > y_1$

$r_1 \leftarrow x_1$

@ $r_1 = \max\{x_1, y_1\}$

assume $r_1 = \max\{x_1, y_1\}$

assume $x_2 > y_2$

assume $\neg[x_1 > y_1]$

$r_1 \leftarrow y_1$

@ $r_1 = \max\{x_1, y_1\}$

...

$r_2 \leftarrow x_2$

@ $r_1 = \max\{x_1, y_1\}, r_2 = \max\{x_2, y_2\}$

2 $\times$ 3 = 6 basic paths

in addition: shorter, simpler

Basic Paths of **if-then-else** Cascades

```
@  
if  $P_1$  then  
    ...  
else  
    ...  
...  
if  $P_n$  then  
    ...  
else  
    ...  
@
```

Result: 2^n basic paths

Basic Paths of **if-then-else** Cascades

@

if P_1 **then**

...

else

...

...

if P_n **then**

...

else

...

@

after introducing intermediate assertions: $2n$ basic paths

Loops

$i \leftarrow 0$

while $a[i] \neq 0$ **do**

$i \leftarrow i + 1$

@ $a[i] = 0 \wedge \forall k \in \{0, \dots, i - 1\} . a[k] \neq 0$

Basic paths:

$i \leftarrow 0$

assume $a[i] \neq 0$

$i \leftarrow i + 1$

assume $a[i] \neq 0$

$i \leftarrow i + 1$

assume $a[i] \neq 0$

$i \leftarrow i + 1$

...

assume $a[i] = 0$

@ $a[i] = 0 \wedge \forall k \in \{0, \dots, i - 1\} . a[k] \neq 0$

Problem: **infinitely many** basic paths! repeating parts

Again: intermediate assertions

Loops

```
 $i \leftarrow 0$   
while  $a[i] \neq 0$  do  
  @  
   $i \leftarrow i + 1$   
  @  $a[i] = 0 \wedge \forall k \in \{0, \dots, i - 1\} . a[k] \neq 0$ 
```

Intermediate assertions:

```
 $i \leftarrow 0$   
assume  $a[i] \neq 0$   
@  
 $i \leftarrow i + 1$   
assume  $a[i] \neq 0$   
@  
 $i \leftarrow i + 1$   
assume  $a[i] \neq 0$   
@  
 $i \leftarrow i + 1$   
...  
assume  $a[i] = 0$   
@  $a[i] = 0 \wedge \forall k \in \{0, \dots, i - 1\} . a[k] \neq 0$ 
```

Loops

```
 $i \leftarrow 0$   
while  $a[i] \neq 0$  do  
   $@ \forall k \in \{0, \dots, i\} . a[k] \neq 0$   
   $i \leftarrow i + 1$   
 $@ a[i] = 0 \wedge \forall k \in \{0, \dots, i - 1\} . a[k] \neq 0$ 
```

Basic paths:

```
 $i \leftarrow 0$   
assume  $a[i] = 0$   
 $@ a[i] = 0 \wedge \forall k \in \{0, \dots, i - 1\} . a[k] \neq 0$ 
```

```
 $i \leftarrow 0$   
assume  $a[i] \neq 0$   
 $@ \forall k \in \{0, \dots, i\} . a[k] \neq 0$ 
```

```
assume  $\forall k \in \{0, \dots, i\} . a[k] \neq 0$   
 $i \leftarrow i + 1$   
assume  $a[i] \neq 0$   
 $@ \forall k \in \{0, \dots, i\} . a[k] \neq 0$ 
```

```
assume  $\forall k \in \{0, \dots, i\} . a[k] \neq 0$   
 $i \leftarrow i + 1$   
assume  $a[i] = 0$   
 $@ a[i] = 0 \wedge \forall k \in \{0, \dots, i - 1\} . a[k] \neq 0$ 
```

Linearization of **while**

Assumption: loop contains at least one assertion @ (a *loop invariant*)

while P **do**

...

@

...

Instead of **while** P **do**:

- ▶ on every basic path entering or re-entering the loop: **assume** P
- ▶ on every basic path leaving or jumping over loop: **assume** $\neg P$

If a loop contains one assertion and no further control structures:
4 resulting basic paths (see “Linearization of Control Structures”)

further control structures?

Linearization of **for** Loops

```
assume  $k \geq 5$   
for  $i \leftarrow 1$  to 10 do  
  @  $k \geq 3$   
   $k \leftarrow k + i$   
@  $k \geq 0$ 
```

```
assume  $k \geq 5$   
assume  $\neg 1 \leq 10$   
@  $k \geq 0$ 
```

```
assume  $k \geq 5$   
assume  $i = 1$   
@  $k \geq 3$ 
```

```
assume  $1 \leq i \leq 10 \wedge k \geq 3$   
 $k \leftarrow k + i$   
 $i \leftarrow i + 1$   
assume  $i \leq 10$   
@  $k \geq 3$ 
```

```
assume  $1 \leq i \leq 10 \wedge k \geq 3$   
 $k \leftarrow k + i$   
 $i \leftarrow i + 1$   
assume  $\neg i \leq 10$   
@  $k \geq 0$ 
```

Corresponding Verification Conditions

assume $1 \leq i \leq 10 \wedge k \geq 3$

$k \leftarrow k + i$

$i \leftarrow i + 1$

assume $i \leq 10$

@ $k \geq 3$

assume $1 \leq i \leq 10 \wedge k \geq 3$

$k_1 \leftarrow k + i$

$i_1 \leftarrow i + 1$

assume $i_1 \leq 10$

@ $k_1 \geq 3$

Verification condition:

$$[1 \leq i \wedge i \leq 10 \wedge k \geq 3 \wedge k_1 = k + i \wedge i_1 = i + 1 \wedge i_1 \leq 10] \Rightarrow k_1 \geq 3$$

Linearization of **for**

for $i \leftarrow l$ **to** u **do**

...

@

...

Assumptions:

- ▶ loop contains at least one assertion @
- ▶ loop body does not modify i

Instead of **for** ...:

- ▶ path jumping over loop: **assume** $\neg l \leq u$
- ▶ path into loop from outside: **assume** $i = l \wedge l \leq u$
- ▶ path re-entering loop: $i \leftarrow i + 1$; **assume** $i \leq u$
- ▶ path leaving loop, depending on programming language
 - ▶ $i \leftarrow i + 1$; **assume** $\neg i \leq u$ (e.g., C)
 - ▶ in some programming languages: **assume** $i = u$ (e.g., Pascal)

Moreover: We may additionally assume $l \leq i \leq u$
at the beginning of any path starting inside of the loop

Linearization of Control Structures

A basic path is a *basic path of a program* P iff the basic path

- ▶ corresponds to a sequence of program lines of P that can be executed in this order,
- ▶ either begins at the beginning of P , or at an @-assertion that is replaced by the corresponding **assume**-assertion, and
- ▶ contains **assume** ϕ instead of control structures, where ϕ is the condition under which program execution follows this path.

If for all basic paths π of a program P , $\models VC(\pi)$, then the program is correct.

A program with loops may have **infinitely** many basic paths and corresponding verification conditions

Program Correctness and Loops

How to ensure a finite number of basic paths?

Every **loop** must contain at least **one assertion**
(*loop invariant*)

The loop invariant can be at any position in the loop

Nested loops: every loop needs an assertion, not just the innermost one!

Preliminary summary:

- loop invariants $\Rightarrow$

- finite number of basic paths $\Rightarrow$

- finite number of verification conditions $\Rightarrow$

- we can check program correctness

General Program Linearization

Example:

if C_1 **then**

$P_1 \dots$

else

$P_2 \dots$

while C_2 **do**

$@ A_1$

$P_3 \dots$

while C_3 **do**

$@ A_2$

$P_4 \dots$

$P_5 \dots$

$P_6 \dots$

$@ A_3$

basic paths:

assume C_1 ; P_1 ; **assume** C_2 ; $@ A_1$

assume C_1 ; P_1 ; **assume** $\neg C_2$; P_6 ; $@ A_3$

assume $\neg C_1$; P_2 ; **assume** C_2 ; $@ A_1$

assume $\neg C_1$; P_2 ; **assume** $\neg C_2$; P_6 ; $@ A_3$

assume A_1 ; P_3 ; **assume** C_3 ; $@ A_2$

assume A_1 ; P_3 ; **assume** $\neg C_3$; P_5 ; **assume** C_2 ; $@ A_1$

assume A_1 ; P_3 ; **assume** $\neg C_3$; P_5 ; **assume** $\neg C_2$; P_6 ; $@ A_3$

assume A_2 ; P_4 ; **assume** C_3 ; $@ A_2$

assume A_2 ; P_4 ; **assume** $\neg C_3$; P_5 ; **assume** C_2 ; $@ A_1$

assume A_2 ; P_4 ; **assume** $\neg C_3$; P_5 ; **assume** $\neg C_2$; P_6 ; $@ A_3$

Verification Conditions Hold vs. No Assertion Failure

```
 $i \leftarrow 1$   
while  $\top$  do  
   $@ i \geq -1$   
   $i \leftarrow 2i$ 
```

Does the **assertion** hold during program execution? **yes**

Do all **verification conditions** hold? **No!!!**

The verification condition of the basic path

```
assume  $i \geq -1$ 
```

```
 $i \leftarrow 2i$ 
```

is

$$[i \geq -1 \wedge i_1 = 2i] \Rightarrow i_1 \geq -1$$

```
assume  $\top$ 
```

```
 $@ i \geq -1$ 
```

which does not hold. Counter-example: $\{i \leftarrow -1; i_1 \leftarrow -2\}$

Verification Conditions Hold vs. No Assertion Failure

If for all basic paths π of a program P , $\models VC(\pi)$,
then the program is correct.

Difference:

- ▶ no assertion failure: **global** condition
(for checking, one needs to understand the whole program)
- ▶ verification conditions hold: **locally checkable** condition

In practice:

- ▶ manually checking all VCs usually unrealistic
- ▶ make assertions as locally checkable as possible

Program Correctness and Loops: Terminology

A loop invariant is *correct* iff it holds for all executions
(no assertion failure)

Problem: this depends on the *global* program behavior

A loop invariant is *inductive* iff
all verification conditions ending in this loop invariant are true

inductiveness implies correctness, but not vice versa

Main remaining question:

How to come up with inductive loop invariants?

Finding Inductive Loop Invariants: Example

Specification:

- ▶ Input: array a
- ▶ Output: r s.t. $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$

Program:

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
    if  $a[i] = 0$  then  $r \leftarrow \top$   
@  $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$   
return  $r$ 
```


Inductive Loop Invariant for Example

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
  @  $\top$   
  if  $a[i] = 0$  then  $r \leftarrow \top$   
  @  $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$   
return  $r$ 
```

1			4
4	3	0	6

i	r
1	$\perp$
2	$\perp$
3	$\perp$
4	$\top$

In general:

Finding inductive invariants is an art, no technique can replace ideas!

Inductivity ensured by three verification conditions:

- ▶ holds in **first** loop iteration
- ▶ if it holds, and the loop is **re-entered**, then it must hold **again**
- ▶ if it holds, and the loop is **left**, then assertion **after** the loop must hold

Guess $\top$

Inductive Loop Invariant for Example

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
  @  $\top$   
  if  $a[i] = 0$  then  $r \leftarrow \top$   
@  $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$   
return  $r$ 
```

Path leaving loop through else branch:

```
assume  $\top$   
assume  $a[i] \neq 0$   
assume  $i = n$   
@  $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$ 
```

Counterexample:

1		4	
0	0	0	6

i	r
4	$\perp$

Guess must be strengthened

Guess $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$

Checking Inductivity of the Invariant

$r \leftarrow \perp$

for $i \leftarrow 1$ **to** n **do**

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$

if $a[i] = 0$ **then** $r \leftarrow \top$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$

return r

¹			⁴
4	3	0	6

i	r
1	$\perp$
2	$\perp$
3	$\perp$
4	$\top$

Paths through loop:

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$

assume $a[i] = 0$

$r \leftarrow \top$

$i \leftarrow i + 1$

assume $i \leq n$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$

assume $a[i] \neq 0$

$i \leftarrow i + 1$

assume $i \leq n$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$

Checking Inductivity of the Invariant

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n$  do  
   $@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$   
  if  $a[i] = 0$  then  $r \leftarrow \top$   
 $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$   
return  $r$ 
```

Final basic paths:

```
assume  $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$   
assume  $a[i] = 0$   
 $r \leftarrow \top$   
assume  $i = n$   
 $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$ 
```

```
assume  $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$   
assume  $a[i] \neq 0$   
assume  $i = n$   
 $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$ 
```

Loop Invariants and Program Development

Better: develop loop invariant **before** loop body

Specification:

- ▶ Input: array a
- ▶ Output: r s.t. $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$

$r \leftarrow \perp$

for $i \leftarrow 1$ **to** n **do**

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$

 ???**if** $a[i] = 0$ **then** $r \leftarrow \top$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq i \wedge a[k] = 0]$

$@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$

return r

Programs with Bugs

Example:

```
 $r \leftarrow \perp$   
for  $i \leftarrow 1$  to  $n - 1$  do  
   $@ r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$   
  if  $a[i] = 0$  then  $r \leftarrow \top$   
 $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$   
return  $r$ 
```

Final basic paths:

```
assume  $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$   
assume  $a[i] = 0$   
 $r \leftarrow \top$   
assume  $i = n - 1$   
 $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$ 
```

```
assume  $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$   
assume  $a[i] \neq 0$   
assume  $i = n - 1$   
 $@ r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$ 
```

Basic Path: **if**

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$

assume $a[i] = 0$

$r \leftarrow \top$

assume $i = n - 1$

@ $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$

Under the given assumptions, both sides of **equivalence hold**.

The basic path is correct.

Intuition: after execution of the **if** branch,
the result is correct (independently of the initial state)

Basic Path: **else**

assume $r \Leftrightarrow [\exists k . 1 \leq k \leq i - 1 \wedge a[k] = 0]$

assume $a[i] \neq 0$

assume $i = n - 1$

@ $r \Leftrightarrow [\exists k . 1 \leq k \leq n \wedge a[k] = 0]$

Assumptions do not constrain $a[n]$

In general, not correct

Automation

Explicit support for writing and checking assertions is **growing**, for example

- ▶ Eiffel
- ▶ Microsoft Code Contracts
- ▶ ANSI/ISO C Specification Language
- ▶ Java Modeling Language (JML), OpenJML (<http://www.openjml.org>)
- ▶ KeY (<http://www.key-project.org>)
- ▶ Dafny (<http://www.dafny.org>) (used by Amazon)
- ▶ TLA+ (Microsoft, also used by Amazon)
- ▶ ...

The resulting **verification conditions** can often be **proved automatically**.

Remaining problem: we **need** loop **invariants**

Automated Finding of Inductive Loop Invariants

Examples of industrial tools:

- ▶ <http://www.absint.com/astree/>
- ▶ Mathworks Polyspace
- ▶ <https://www.imandra.ai>

Finite state systems: viz MI-TES

Summary: Assertions for Program Correctness

For proving correctness of a program we need

1. at least one assertion in every loop, that is, **loop invariants**
2. a proof of the **verification condition** of every basic path of the program

Conclusion

In practice, usually **no** correctness **proof** needed

Still, writing assertions has many **advantages**

Localize understanding of program:

instead of thinking about correctness of whole program

just think about correctness **from one assertion to the next**

... even if incomplete, and unproved

Systematic approach to code reviews

Assertions represent the **essence of correctness** of a program:

- ▶ important documentation
- ▶ check correctness during program execution
- ▶ more and more: (semi)-automatic correctness proofs based on assertions

Literature I

Aaron Bradley and Zohar Manna. *The Calculus of Computation*. Springer, 2007.